

# Strings

Eunji Kim

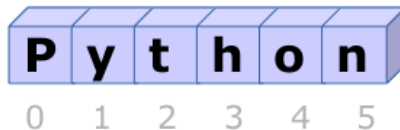
[eunjikim@cau.ac.kr](mailto:eunjikim@cau.ac.kr)

# String

In Python, a string is a sequence of characters.

Strings are surrounded by either single quotation marks, or double question marks.

- `'Python'` is the same as `"Python"`.
- `Python` is a string made of 6 characters: `P`, `y`, `t`, `h`, `o`, and `n`.



## Quotes Inside Quotes

You can use quotes inside a string, as long as they don't match the quotes surrounding the string:

```
print("It's alright") # an error occurs if the surrounding quotes are ``  
print("He is called 'Johnny'")  
print('He is called "Johnny"')
```

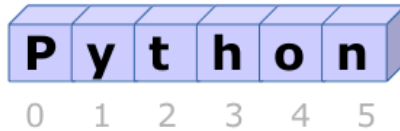
## Multiline Strings

You can assign a multiline string to a variable by using three quotes:

```
a = """Lorem ipsum dolor sit amet,  
consectetur adipiscing elit,  
sed do eiusmod tempor incididunt  
ut labore et dolore magna aliqua."""
```

# Indexing and Slicing

Indexing and slicing can also be applied to a string, just like in a list or tuple:



```
mystr = "Python"
print(len(mystr)) # 6

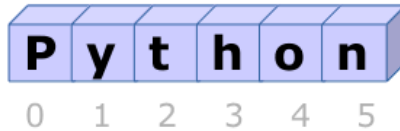
print(mystr[2]) # t
print(mystr[2:5]) # tho
```

A strings is **immutable** like a tuple, so you cannot change its values:

```
mystr[-1] = 'm' # TypeError: 'str' object does not support item assignment
```

# Indexing and Slicing

To access a part of a string, indexing and slicing can be applied using brackets `[]`.



```
mystr = "Python"

# Indexing: obtain individual elements
print(mystr[2]) # t

# Slicing: obtain a sequence of elements
print(mystr[2:5]) # tho
```

 Index starts with `0`.

 We can check the length of a string using `len()`

```
>>> len(mystr)
6
```

# Built-in Methods for Strings

Python has a set of built-in methods that you can use on strings.

The `upper()` and `lower()` methods return the string in uppercase and lowercase, respectively:

```
a = "Hello, World!"  
print(a.upper()) # HELLO, WORLD!  
print(a.lower()) # hello, world!
```

The `replace()` method replaces a string with another string:

```
a = "Hello, World!"  
print(a.replace("H", "J")) # JELLO, WORLD!
```

Whitespace is the space before and/or after the actual text, and very often you want to remove this space.

The `strip()` method removes any whitespace from the beginning or the end:

```
a = '          hello... world ...'  
print(a.strip()) # 'hello... world ...'
```

The `split()` method returns a list where the text between the specified separator ( , , in this case) becomes the list items.

```
a = 'i,my,me,mine'  
print(a.split(',')) # ['i', 'my', 'me', 'mine']
```

# String Concatenation

To concatenate, or combine, two strings you can use the `+` operator.

```
a = "Hello"  
b = "World"  
c = a + b  
print(c) # "HelloWorld"
```



To add a space between them, add a " ":

```
a = "Hello"  
b = "World"  
c = a + " " + b  
print(c) # "Hello World"
```

Or, you can use `join()` method:

```
a = "Hello"  
b = "World"  
c = " ".join([a, b])  
print(c) # "Hello World"
```

# String Format

You can format a string in Python using several methods: f-strings, the `.format()` method, and the modulo operator ( `%` ).

```
adj = "Red"
noun = "Alert"

cheese = "%s %s" % (adj, noun) # This modulo style was deprecated (PEP 3101)
print(cheese) # Red Alert

cheese = "{} {}".format(adj, noun) # Possible since Python 3
print(cheese) # Red Alert

cheese = "{0} {1} {1} {0}".format(adj, noun) # Numbers can also be reused
print(cheese) # Red Alert Alert Red

cheese = "{adj} {noun}".format(adj=adj, noun=noun) # using keyword arguments
print(cheese) # Red Alert
```

We cannot combine strings and numbers like this:

```
age = 36
txt = "My name is John, I am " + age + " years old"
print(txt) # TypeError: can only concatenate str (not "int") to str
```

To combine strings and numbers, you must cast numbers into strings:

```
age = 36
txt = "My name is John, I am " + str(age) + " years old"
print(txt) # My name is John, I am 36 years old
```

F-string is the preferred way to combine strings and numbers.

To specify a string as an f-string, simply put an `f` in front of the string literal, and add curly brackets `{}` as *placeholders* for variables and other operations.

Create an f-string:

```
age = 36
txt = f"My name is John, I am {age} years old."
print(txt) # My name is John, I am 36 years old.
```

A placeholder can include a **modifier** to format the value.

A modifier is included by adding a colon : followed by a legal formatting type, like .2f which means fixed point number with 2 decimals:

```
price = 59.1254
txt = f"The price is {price:.2f} dollars"
print(txt) # "The price is 59.12 dollars"
```

A placeholder can contain Python code, like math operations:

```
txt = f"The price is {20 * 30} dollars"
print(txt) # "The price is 600 dollars"
```

# Escape Character

To insert characters that are illegal in a string, use an escape character.

An escape character is a backslash `\` followed by the character you want to insert.

For example, you will get an error if you use double quotes inside a string that is surrounded by double quotes:

```
txt = "We are the so-called "Vikings" from the north." # SyntaxError: invalid syntax
```

To fix this problem, use the escape character `\` in front of `"`:

```
txt = "We are the so-called \"Vikings\" from the north."
```

Or, just use `'` instead of `"` to surround the string:

```
txt = 'We are the so-called "Vikings" from the north.'
```

# Frequently Used Escape Characters

`\n` for new line:

```
>>> x = 'i\nmy\nme\nmine'
>>> x
i
my
me
mine
```

`\t` for tab:

```
>>> x = 'i\tmy\tme\tmine'
>>> x
i    my    me    mine
```

**Thank You.**